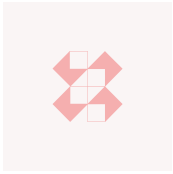

Xenia

tip the post, not the platform.

On-chain tipping for X (Twitter), built on Somnia Network. Tip anyone by their handle in one click — sub-second finality, sub-cent gas, and an on-chain escrow that lets you tip people before they even have a wallet.

SUBMISSION DOCUMENT · LIVE ON SOMNIA TESTNET

Contents

01 Overview

Xenia turns every post on **X (Twitter)** into a tippable on-chain action. A Chrome extension injects a **Tip** button into the X feed and onto profiles; clicking it triggers a transaction on **Somnia Network** that either lands instantly in the recipient's wallet or is held in an on-chain **escrow** keyed by the recipient's handle. When that recipient later signs up, the backend binds their wallet on-chain and they claim every tip waiting for them with a single click.

The entire stack — two Solidity contracts, a Node/Express backend with a Postgres index, a React dashboard, a browser extension, and an AI command bot — is built specifically around Somnia's strengths. Sub-second finality and sub-cent gas are not a nice-to-have here; they are the precondition that makes per-post micro-tipping economically and experientially viable for the first time. On a slower or more expensive chain, a $0.10 tip makes no sense: the gas eats the value and the confirmation latency kills the gesture. On Somnia, tipping collapses into a single in-feed tap that settles before the toast notification fades.

Two complementary flows share one contract. In **Mode A**, the user's own wallet signs the tip directly from the extension or the web app. In **Mode B**, the user pre-funds a delegated balance and then tips by tweeting a natural-language command that an AI bot parses and executes on their behalf — without ever being able to withdraw their funds. Both paths converge on the same escrow, the same handle-based identity, and the same one-click claim.

In one sentence: tip anyone on X by their handle, instantly and on-chain, even if they have never touched crypto.

02 The problem

Tipping creators on social media is broken in two opposite directions, and neither serves a casual, in-the-moment gesture.

Web2 tipping

- Both the sender and the creator must create accounts on the same platform.
- The platform takes anywhere from 2% to 30% of every tip.
- Payouts are batched and arrive days later, behind minimums and KYC.
- Value is locked inside one walled garden and cannot move.

Web3 tipping

- The recipient must already hold a wallet on the exact right chain.
- Gas frequently costs more than the tip itself.
- There is no way to reward an identity that is still off-chain.
- Multi-second confirmation latency turns a gesture into a transaction.

The compound result is that tipping is not the fluid, in-feed reflex it should be. It is a friction-filled chore involving sign-ups, fees, address lookups, network switches, and waiting — so almost no one actually does it, and creators capture almost none of the appreciation their work generates.

03 The solution

Xenia removes both walls at once: the wallet wall on the recipient side, and the friction wall on the sender side.

- **Tip anyone, even non-crypto users.** A tip to an unregistered handle is held in an on-chain escrow keyed by that handle. The recipient is not required to do anything in advance. When they

eventually sign up, the backend binds their handle to their wallet on-chain and the escrowed balance becomes claimable — no pre-existing wallet, no chain, no setup.

> **Tipping is one click, inside X.** The extension injects a Tip button into every tweet and onto profiles. Before any transaction it auto-adds Somnia to the user's wallet, so a first-time tipper never has to find an RPC URL or a chain ID.

> **Two modes, one contract.** Mode A is a direct, wallet-signed tip. Mode B lets a user tip by tweeting a command that an AI bot executes from a pre-authorized deposit — useful for power users and for tipping without leaving the timeline.

> **Handle-first identity.** Every flow is keyed by the lowercased X handle, the only identifier a sender reliably knows at tip time. This single decision is what makes "tip someone who isn't here yet" actually work.

04 Identity model — handle-first

The central design decision in Xenia is that the on-chain escrow is keyed by the **lowercased X handle** (for example `vitalik`), not by a numeric account id and not by a wallet address. The reasoning is direct: at the moment a tip is sent, the only thing the sender knows about an unregistered recipient is their handle. A numeric Twitter id is not visible in the feed, and the recipient has no wallet yet by definition.

Keying by handle makes the entire lifecycle internally consistent. The same lowercased handle is used by `tip()` when funds are escrowed, by `registerWallet()` when the recipient binds their wallet, and by `claim()` when they withdraw. Every surface that produces a tip — the web app, the extension, and the AI bot — normalizes the handle the same way (strip a leading `@`, lowercase) so a tip from any source lands in exactly the bucket the recipient later claims from. An earlier iteration keyed sends by handle but claims by numeric id; that mismatch silently made escrowed tips unclaimable, and resolving it is what unified the model described here.

05 How it works

Mode A — direct tip (extension / web)

The user clicks Tip on a tweet or a profile. Their own wallet — a Privy embedded wallet on the web app, or MetaMask in the extension — signs `Escrow.tip(handle)` with the tip amount attached as value. If the handle is already registered, the contract transfers the funds directly to the recipient's wallet in under a second. If it is not, the contract pushes the tip into a per-handle escrow array and emits a `TipSent` event carrying the escrow index; the backend reads that receipt and indexes a pending claim so the recipient will see it the moment they join.

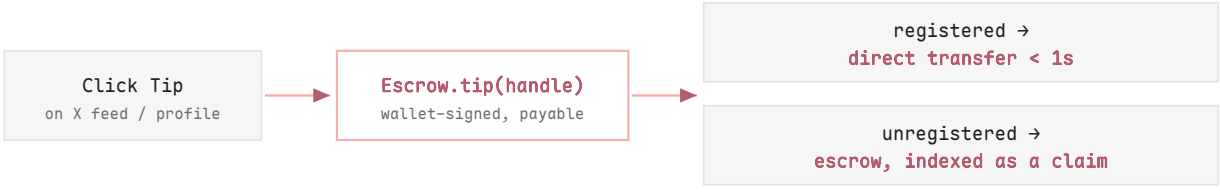

FIGURE 1 — MODE A DIRECT TIP

Mode B — Twitter command

A user who wants to tip without leaving the timeline first calls `deposit()` to fund an internal balance, then `authorize(bot)` to let the Xenia bot spend from it. They then tweet a natural-language command such as `@xeniaBot tip @bob 5 STT`. The bot reads the mention, parses the intent with Gemini

(with a regex fallback if the model is unavailable), resolves the recipient, and calls `tipOnBehalf(sender, handle, amount)`. The contract verifies the authorization, debits the sender's deposit, and either pays the recipient directly or escrows the tip — the bot can only forward funds to the resolved recipient and can never withdraw to an arbitrary address.

FIGURE 2 — MODE B TWITTER COMMAND

Claim

When a recipient who was tipped into escrow finally signs up, the backend calls `registerWallet(handle, wallet)` idempotently — it first checks whether the handle is already bound, and only writes (and waits for the receipt) when it is not, so a claim never races an unbound state. The recipient then calls `claim(handle)` from their own wallet, which pulls the entire pending balance in a single transaction.

FIGURE 3 — CLAIM FLOW

06 Architecture

One escrow contract on Somnia is reached from four surfaces through a single backend. The split of responsibilities is deliberate: the user's wallet signs everything that moves value, the backend wallet (which is the contract owner) only signs the two owner-restricted bindings, and the bot signs only delegated forwards.

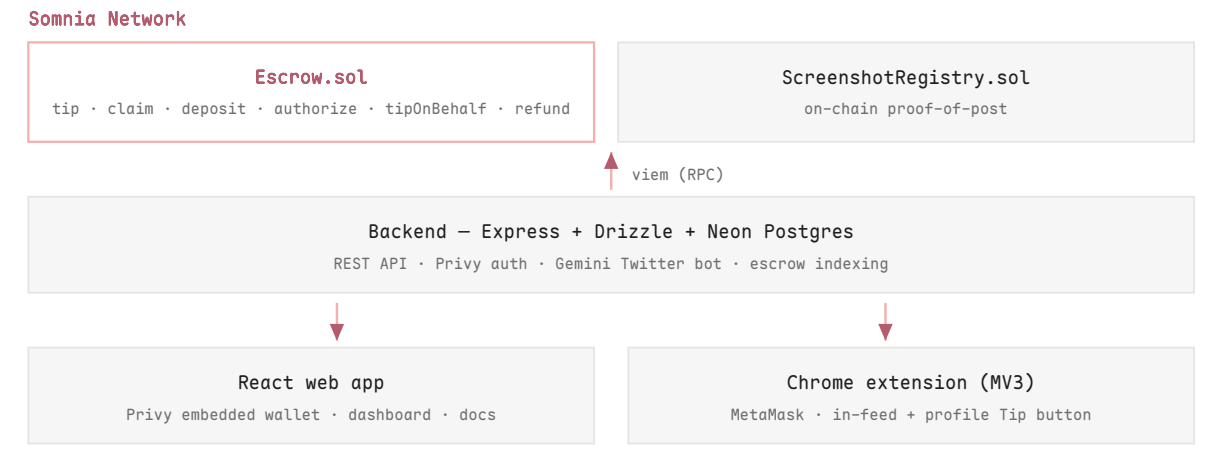

FIGURE 4 — SYSTEM ARCHITECTURE

The web app and the extension both read the active contract address and the bot address from a single `/api/somnia/network` endpoint, so there is one source of truth for deployment configuration. The extension additionally ships a hard-coded fallback escrow address, which means a tip can still be signed and broadcast even if the backend is briefly unreachable.

07 Smart contracts

Escrow.sol

A single contract handles both tipping modes and the full lifecycle of unclaimed funds. Escrowed tips live in a per-handle array; registrations bind a handle to a wallet exactly once; Mode B funds live in a separate per-user deposit balance with an explicit delegate authorization map.

FUNCTION	MODE	DESCRIPTION
`tip(handle)` payable	A	Direct transfer if the handle is registered, otherwise escrow under the handle and emit TipSent.
`deposit()` payable	B	Credit the caller's internal balance for later delegated tipping.
`authorize(delegate)`	B	Allow a delegate (the bot) to call tipOnBehalf for the caller; revocable.
`tipOnBehalf(sender, handle, amount)`	B	Delegate-only; debit sender's deposit and forward to the resolved recipient.
`claim(handle)`	—	The registered wallet pulls all pending escrow tips for the handle.
`refund(handle, idx)`	—	The original sender reclaims a specific tip after a 90-day window if never claimed.
`registerWallet(handle, wallet)`	—	onlyOwner; binds a handle to a wallet, immutable once set.
`withdrawDeposit / deauthorize / setFee / withdrawFees`	—	User and admin housekeeping; fee capped at 5%.

ScreenshotRegistry.sol

A lightweight on-chain proof-of-post that anchors a screenshot CID and tweet id with a timestamp and recorder: `registerScreenshot(cid, tweetId)` (owner-only), `verifyScreenshot(cid)`, and `getProofByTweetId(tweetId)`. It exists to give tipped posts an immutable record, which is only affordable at Somnia's gas levels.

```
Escrow      0×Ef0ca54F3C195737880127df62069C5B5A17B458
Registry    0×9C3c6b9cc4ECdA73e65A240DD0cD075ce202AfE3
Chain       Somnia Shannon Testnet · chainId 50312 · STT
```

08 Backend & API

The backend is a Node + Express service in TypeScript, using Drizzle ORM over a Neon-compatible Postgres database and viem as the chain client. Authentication is handled by Privy: the frontend

attaches a Privy access token as a bearer header, the backend verifies it, and upserts a user row. The chain client is configured for both Somnia testnet (50312, STT) and mainnet (50313, SOMI), with the active chain chosen from the environment at boot.

A key backend responsibility is **escrow indexing**. When a tip is confirmed, the backend reads the transaction receipt, decodes the `TipSent` event, and — only for tips that actually went to escrow — writes an idempotent pending-claim row keyed on the transaction hash and escrow index. The Mode B bot reports its own escrow tips through a dedicated endpoint so that bot-driven tips appear in the recipient's claim list too.

METHOD · PATH	PURPOSE
POST /api/tips/send	Record a tip and return its row id.
POST /api/tips/:id/confirm	Attach the tx hash and index the escrow claim.
GET /api/claims	The signed-in user's pending claims.
GET /api/somnia/network	Chain, contract addresses, and bot address.
POST /api/somnia/register	Idempotent registerWallet for the current user.
GET /api/somnia/balance/:address	Native STT/SOMI balance.
POST /api/proof/register · GET /api/proof/:tweetId	On-chain proof-of-post register and lookup.
POST /api/bot/record-claim · /api/bot/*	Bot-key-authenticated escrow reporting and notifications.

09 Frontend & extension

Web app

The dashboard is a React 18 + Vite application styled with Tailwind in a warm monospace "terminal" design system. Authentication and embedded wallets come from Privy, configured with Somnia as the default chain. Users can send single and batch tips, deposit and authorize the bot for Mode B, view their pending claims and on-chain transaction history, and read a built-in documentation page. In production the same Express server serves this built client, so the whole product runs from a single origin and a single deploy.

Chrome extension (MV3)

The extension injects a Tip button into tweets and onto profile headers on x.com and twitter.com. Because content scripts cannot see the page's injected wallet, a small page-world bridge relays requests to MetaMask over `postMessage`; before every transaction it ensures the wallet is on Somnia, adding the network automatically on first use. The tip is signed by the user's own wallet, broadcast on-chain, and then reported to the backend best-effort — so a tip succeeds even when the user has never opened the web app.

10 The autonomous agent

Mode B is driven by an autonomous agent — a Python service that turns a tweet into an on-chain action. It is provider-agnostic behind a single interface: by default it uses Google Gemini to parse a free-form tip command into structured JSON, falls back to Anthropic Claude if configured, and finally to a deterministic regex matcher if no model is available — so the bot degrades gracefully and never crashes on a parsing failure. It also includes a price helper (so a user can

tip a dollar amount), basic fraud checks (it refuses self-tips), and on-chain guards (it verifies the sender's deposit and the bot's authorization before sending). After a successful tipOnBehalf, the bot inspects the receipt for an escrow event and, if present, reports it to the backend so the recipient can claim.

11 Why Somnia

Sub-second finality and sub-cent gas are precisely the properties that make per-post micro-tipping possible at all, rather than a marginal improvement over an existing flow.

PROPERTY	TYPICAL L1/L2	SOMNIA	WHY IT MATTERS FOR XENIA
Finality	2–12s	< 1s	In-feed tipping feels like Web2 — no spinner.
Gas / tip	$0.05–$1.00	< $0.001	$0.10–$1.00 tips are economically sane.
Throughput	100–4,000 TPS	1,000,000+ TPS	Headroom for a viral app, no fee spikes.
EVM	Yes	Yes	Solidity + Hardhat + viem port cleanly.

Xenia was originally conceived on a slower chain. The migration to Somnia is what turns the gesture from "I am sending a transaction" into "I just tapped a button" — and it is also what makes anchoring an on-chain proof-of-post on every interaction cheap enough to be worth doing.

12 Security

- **Reentrancy guard** on every external call that moves value (tip, claim, refund, deposit, withdraw, tipOnBehalf, withdrawFees).
- **Fee accounting is separated** from user funds: accumulated fees live in their own variable, so a fee withdrawal can never touch a user balance.
- **Wallet binding is immutable** — a handle can be bound to a wallet exactly once and never rebound.
- **Bot delegation is scoped to forwarding only** — tipOnBehalf can route only to the resolved recipient, never to an arbitrary address, and the bot can never withdraw.
- **90-day sender refund** protects funds tipped to a handle that never registers.
- **Two-step ownership transfer** on both contracts; the platform fee is capped at 5%.
- **No custody of keys** — Privy embedded wallets stay client-side; the backend never holds user private keys, and bot endpoints require a shared secret.

13 Deployment

Because the Express server serves the built React client in production, the entire application runs from a single Railway service: one build, one start command, one origin — which also removes cross-site cookie and CORS complexity. The database is a managed Neon Postgres instance. A preflight script validates the environment and probes the configured contract addresses on-chain (verifying real bytecode) before any deploy, and the build explicitly installs dev dependencies so the client bundles correctly regardless of the production environment flag. The contracts are already deployed and verified live on Somnia testnet.

14 Status, roadmap & network reference

PHASE	SCOPE
Now	Live on Somnia testnet · Mode A end-to-end · on-chain escrow indexing · extension demo with in-feed and profile tipping.
Next	Mainnet deployment (chainId 50313, SOMI) · Chrome Web Store listing · X login enabled via the Privy dashboard.
Later	Public @xeniaBot launch on a paid X API tier · indexer-backed leaderboards and analytics · third-party contract audit before high-value flows.

PROPERTY	TESTNET	MAINNET
Chain ID	50312	50313
Symbol	STT	SOMI
RPC	dream-rpc.somnia.network	mainnet-rpc.somnia.network
Explorer	shannon-explorer.somnia.network	explorer.somnia.network

xenia // somnia — tip the post, not the platform.